Artificial Intelligence
Lecture 13: Machine Learning and Neurosymbolic AI

Prof. Dr. Pierre-Alexandre Murena

SoSe 2025

Outline

1. From search to reinforcement learning
2. Machine learning and knowledge

Plan

1. From search to reinforcement learning
   - Beyond search: Markov Decision Processes
   - Partial observability in MDPs
   - Monte Carlo Tree Search
2. Machine learning and knowledge

Markov Decision Processes (MDP)

Markov Decision Process

A **Markov Decision Process** (MDP) is given by:

- An **action space** $\mathcal{A}$
- A **state space** $\mathcal{S}$
- A **transition function** $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$
- A **reward function** $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$
- A **discount factor** $\gamma \in [0, 1]$

Remark: This is a probabilistic version of the search problem formulation!

Value function

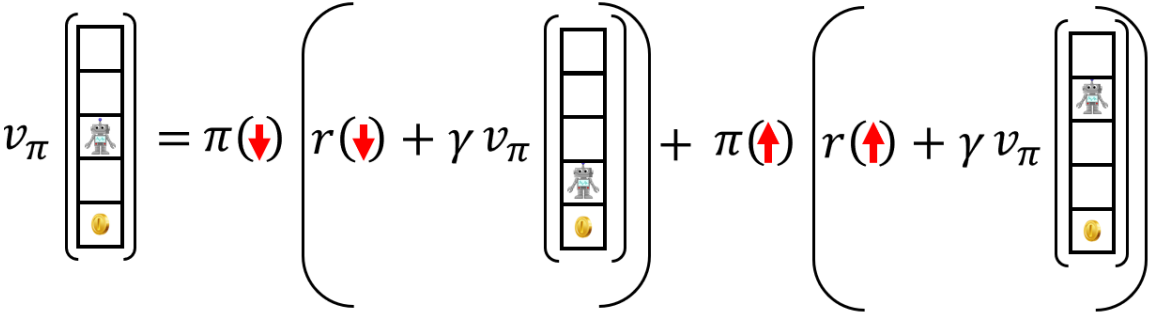
Policy

A **policy** π is a probability distribution over actions conditioned by the state: $\pi(a|s)$ is the probability to pick action a in state s.

We call the **state-value function** of a state s under policy π the quantity:

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \;\middle|\; S_t = s\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right]$$

Computing the value function



Computing the value function

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi\left[G_t \;\middle|\; S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1} \;\middle|\; S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} \;\middle|\; S_t = s\right] + \gamma \mathbb{E}_\pi\left[G_{t+1} \;\middle|\; S_t = s\right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)\left[r + \mathbb{E}_\pi\left[G_{t+1} \;\middle|\; S_{t+1} = s'\right]\right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + v_\pi(s')\right]
\end{aligned}
$$

Bellman equation for state-value function

$$
v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_\pi(s')\right]
$$

Action-value function

We call the **action-value function** of a state s under policy π the quantity:

$$q_\pi(s,a) = \mathbb{E}_\pi\left[G_t \;\middle|\; S_t = s, A_t = a\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s, A_t = a\right]$$

Bellman equation for action-value function

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right]$$

Comparing policies

We can order policies:

$$\pi_1 \geq \pi_2 \;\Leftrightarrow\; v_{\pi_1}(s) \geq v_{\pi_2}(s) \quad \forall s \in \mathcal{S}$$

Comparing policies

We can order policies:

$$\pi_1 \geq \pi_2 \;\Leftrightarrow\; v_{\pi_1}(s) \geq v_{\pi_2}(s) \quad \forall s \in \mathcal{S}$$

We define the optimal state-value function:

$$v^*(s) = \max_{\pi} v_\pi(s) = v_{\pi^*}(s) \qquad \forall s \in \mathcal{S}$$

Comparing policies

We can order policies:

$$\pi_1 \geq \pi_2 \;\Leftrightarrow\; v_{\pi_1}(s) \geq v_{\pi_2}(s) \quad \forall s \in \mathcal{S}$$

We define the optimal state-value function:

$$v^*(s) = \max_{\pi} v_\pi(s) = v_{\pi^*}(s) \qquad \forall s \in \mathcal{S}$$

Theorem

There exists an optimal policy π^* that is at least as good as all other policies:

$$\pi^* \geq \pi \qquad \forall \pi$$

The key problem of RL

How to compute the optimal policy?

Two families of RL problems

In **Model-based RL**, we know the transition and reward functions. The two main methods for model-based RL are:

- Policy iteration: *Starting from an arbitrary policy, iteratively evaluate and improve the policy until convergence.*
- Value iteration: *Compute the optimal state value function by iteratively updating the estimate, and then compute the associated policy*

Two families of RL problems

In **Model-based RL**, we know the transition and reward functions. The two main methods for model-based RL are:

- Policy iteration: *Starting from an arbitrary policy, iteratively evaluate and improve the policy until convergence.*
- Value iteration: *Compute the optimal state value function by iteratively updating the estimate, and then compute the associated policy*

In **Model-free RL**, the transition and reward functions are not known. The value function needs to be estimated using:

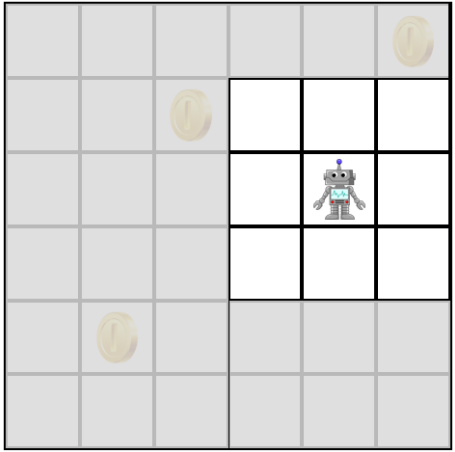
- Monte Carlo policy evaluation: *Sample some trajectories given a policy π.*
- Temporal differences methods: *Similar to MC methods, but don't wait for the end of an episode to update the value.*

Q-learning is then used to find the optimal policy.

Plan

1. From search to reinforcement learning
   - Beyond search: Markov Decision Processes
   - Partial observability in MDPs
   - Monte Carlo Tree Search
2. Machine learning and knowledge

The problem of partial observability



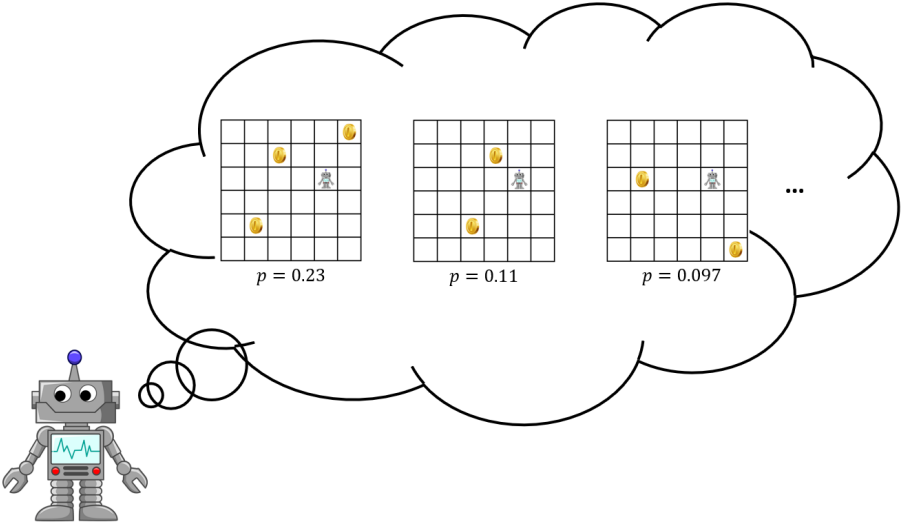
Partially Observable Markov Decision Processes

Partially Observable Markov Decision Process

A **Partially Observable Markov Decision Process** (POMDP) is given by:

- An **action space** $\mathcal{A}$
- A **state space** $\mathcal{S}$
- A **space of observations** $\mathcal{O}$
- A **transition function** $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$
- An **observation function** $\mathcal{O} : \mathcal{A} \times \mathcal{S} \times \Omega \to [0, 1]$
- A **reward function** $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$
- A **discount factor** $\gamma \in [0, 1]$

Belief state MDP



Belief state MDP

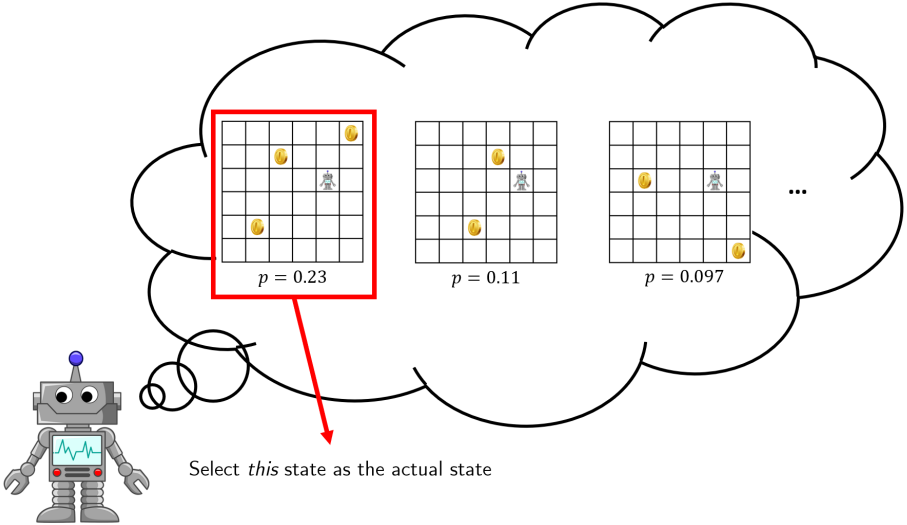
The agent maintains a probability distribution over possible states at each time step:

This probability is updated using Bayes rule, based on the observations:

$$\begin{aligned} P(s_{t+1} \mid a_t, o_{t+1}) &= \alpha P(o_{t+1} \mid a_t, s_{t+1}) P(s_{t+1} \mid a_t) \\ &= \alpha P(o_{t+1} \mid a_t, s_{t+1}) P(s_{t+1} \mid a_t, s_t) P(s_t) \\ &= \alpha \mathcal{O}(o_{t+1} \mid a_t, s_{t+1}) \mathcal{T}(s_{t+1} \mid a_t, s_t) P(s_t) \end{aligned}$$

Introducing probabilistic belief states makes it possible to manage the uncertainty, but **how does this uncertainty impact the choice of actions?**

A solution: Greedy sampling



Greedy sampling and Thompson sampling

The idea of the **greedy** solution is to consider that the current state is the most probable state in the belief state, i.e. the state with the highest probability:

$$\hat{s}_t = \arg\max_s P(s_t)$$

The planning is then done starting from state $\hat{s}_t$ and ignoring the other possibilities.

This solution is not optimal: it ignores most of the states, and gives too much weight to the most probable one.

As a solution, **Thompson sampling** consists in sampling the current state from the probability distribution:

$$\hat{s}_t \sim P(s_t)$$

Plan

1. From search to reinforcement learning
   - Beyond search: Markov Decision Processes
   - Partial observability in MDPs
   - Monte Carlo Tree Search
2. Machine learning and knowledge

Online planning with search

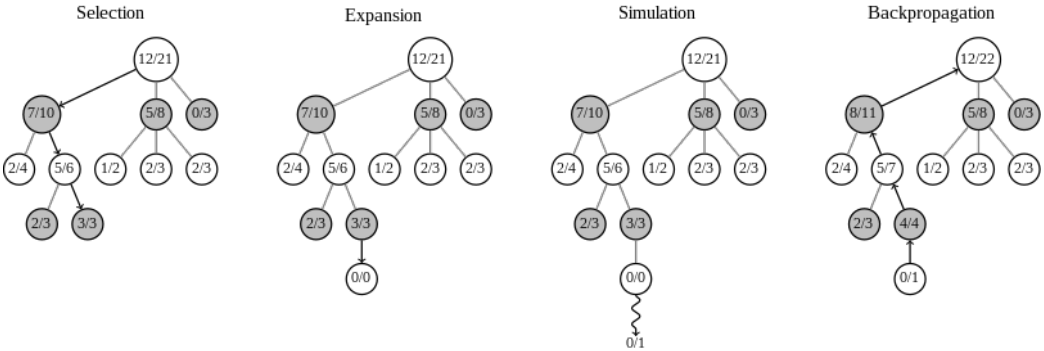
How to choose actions in an MDP?

We need to know the action-value of each action in the current state, in order to select the optimal one.

This is not possible in situations where the state space is too large (for instance 10^{170} states for Go): in such situations it is better to estimate the value **online** and based on the sub-MDP starting from the current state:

1. Build the search tree starting from the current state;
2. Estimate the value of each state using simulated episodes;
3. Play the action with the highest value;
4. Repeat from the new state.

Monte Carlo Tree Search (MCTS)



Step 1: Selection

The selection step corresponds to sampling a trajectory in the nodes we have already seen. This is done using Upper Confidence Bound (UCB).

UCB is a selection strategy that consists in choosing the action when there is uncertainty on its probability. UCB does not consider only the expected value, but also the variance in the observations.

Next steps

Step 2: Expansion

Expansion consists in adding a new node, when we reach the frontier of explored nodes.

Next steps

Step 2: Expansion

Expansion consists in adding a new node, when we reach the frontier of explored nodes.

Step 3: Simulation

Starting from the newly added node, we simulate a trajectory by sampling actions randomly (**rollout**) until we reach a terminal state.

Next steps

Step 2: Expansion
Expansion consists in adding a new node, when we reach the frontier of explored nodes.

Step 3: Simulation
Starting from the newly added node, we simulate a trajectory by sampling actions randomly (**rollout**) until we reach a terminal state.

Step 4: Backpropagation
Backpropagate the value to all the nodes in the trajectory.

Outline

1 From search to reinforcement learning

2 Machine learning and knowledge

What is machine learning?

A historical (but yet still valid) definition of Machine learning (by Tom Mitchell)

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E."

What is machine learning?

A historical (but yet still valid) definition of Machine learning (by Tom Mitchell)

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E."

Example: Image classification:

- Tasks T: classifying images based on their content
- Performance measure P: Proportion of good predictions
- Experience E: dataset of labelled images

Supervised learning

In supervised learning, data are of the form $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, where:

- $x_i \in \mathcal{X}$ are inputs
- $y_i \in \mathcal{Y}$ are outputs

The goal is to be able to predict the correct output y for some future input x. In practice, this is often done by learning a function $h : \mathcal{X} \to \mathcal{Y}$, called **hypothesis**.

Given a **hypothesis class** $\mathcal{H} = \{h : \mathcal{X} \to \mathcal{Y}\}$, we select the one hypothesis h^* satisfying:

$$h^* \in \arg\min_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i) + \mathcal{R}(h)$$

Plan

1 From search to reinforcement learning

2 Machine learning and knowledge
- Integrating knowledge
- Reasoning machines

An example

Consider the task of classifying images:

- Inputs: Photos
- Outputs: Label (e.g. flower, car, dog...)

In this example, the system will learn K predictors $h_1, \ldots, h_K$ predicting whether a given input corresponds to each of the K possible classes. Usually $h_k : \mathcal{X} \to [0, 1]$.

An example

Consider the task of classifying images:

- Inputs: Photos
- Outputs: Label (e.g. flower, car, dog...)

In this example, the system will learn K predictors $h_1, \ldots, h_K$ predicting whether a given input corresponds to each of the K possible classes. Usually $h_k : \mathcal{X} \to [0, 1]$.

We know from the real world that:

$$\forall x \; dog(x) \to animal(x)$$

Solution 1: Data augmentation

Expert knowledge can be used to **augment** the dataset.

Example: If I know that x_i has label "dog", I can also add the point (x_i, "animal") to the dataset.

Pros:

- If the original dataset is consistent, then the augmented dataset is consistent as well
- More data is always welcom

Cons:

- Learning guarantees hold only for i.i.d. data, which is not the case anymore after augmentation
- No guarantee that the learned classifiers satisfy the constraints.

Solution 2: Adapting the learning in consequence

This can be done by:

- Using the logical constraint as a regularizer

Solution 2: Adapting the learning in consequence

This can be done by:

- Using the logical constraint as a regularizer
- Modifying the architecture in accordance to the logical constraint

Solution 2: Adapting the learning in consequence

This can be done by:

- Using the logical constraint as a regularizer
- Modifying the architecture in accordance to the logical constraint
- Modifying the loss in accordance to the logical constraint

Plan

1 From search to reinforcement learning

2 Machine learning and knowledge
- Integrating knowledge
- Reasoning machines

Chomsky vs Norvig

The recent developments of AI oppose two important points of views, embodied by two names:

Noam Chomsky: Intelligence is purely symbolic, and statistical effects are only a superficial:

[I]f you get more and more data, and better and better statistics, you can get a better and better approximation to some immense corpus of text [...] but you learn nothing about the language.

Peter Norvig: Advocates for performance-based models of language, the theory of which is driven by data